

Table of content

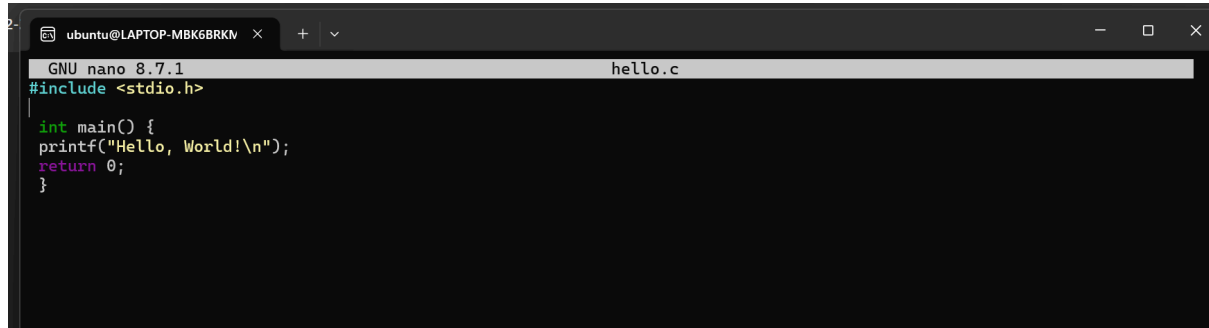
II. C Program Structure and Examples.....	3
A. Basic Hello World Program.....	3
B. Program with User Input.....	4
C. Program with Formatted Output.....	5
III. GCC COMPILATION PROCESS.....	6
A. Installing GCC on Linux.....	6
B. Four Stages of Compilation.....	7
C. COMPILATION STAGE EXAMPLES.....	8
IV. Return Values and Exit Status.....	11
A. Understanding Return Values.....	11
B. Checking Exit Status.....	11
V. KNOWLEDGE CHECK QUESTIONS.....	12

Table of Figures

Figure 1: Basic C language, Hello World.....	3
Figure 2: Use of printf and scanf.....	4
Figure 3: Multiple formats and variables.....	5
Figure 4: Installation of GCC and Version check.....	6
Figure 5: Using 4 distinct stages of C compilation.....	7
Figure 5: Preprocessing Stage.....	8
Figure 6: Compilation Stage.....	9
Figure 7: Assembly Stage.....	10
Figure 8: Linking Stage.....	10
Figure 9: Program with return value.....	11
Figure 10: Checking Program Exit Status.....	11

II. C Program Structure and Examples

A. Basic Hello World Program



```
GNU nano 8.7.1 hello.c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

Figure 1: Basic C language, Hello World

Summary: Using the basic C programming language, wrote code in hello.c using nano hello.c.

B. Program with User Input

```
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ cat task1.c
#include <stdio.h>

int main() {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);
    return 0;
}
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ ./task1
-bash: ./task1: No such file or directory
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ gcc task1.c -o task1
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ ./task1
Enter a number: 12
You entered: 12
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ |
```

Figure 2: Use of printf and scanf

Summary: Wrote code in task1.c using nano and to check that used the cat to see and using gcc method to make the code executable.

C. Program with Formatted Output

```
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ cat task2.c
#include <stdio.h>

int main() {
    char name[50];
    int age;
    float height;

    printf("Enter your name: ");
    scanf("%s", name);

    printf("Enter your age: ");
    scanf("%d", &age);
    printf("Enter your height (m): ");
    scanf("%f", &height);

    printf("\n--- User Info ---\n");
    printf("Name: %s\n", name);
    printf("Age: %d years\n", age);
    printf("Height: %.2f m\n", height);
    return 0;
}
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ |
```

Figure 3: Multiple formats and variables

Summary: Wrote the code in task2.c using nano, using different variables and formatted output.

III. GCC COMPILATION PROCESS

A. Installing GCC on Linux

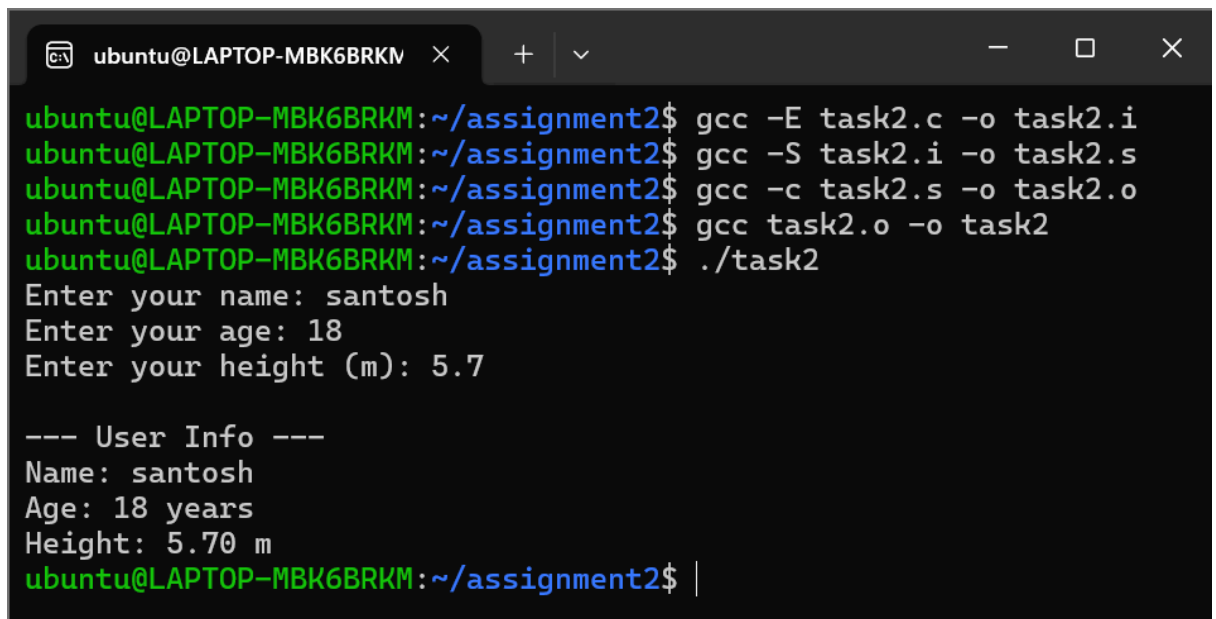
```
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ sudo apt install build-ess
essential
build-essential is already the newest version (12.12ubuntu2.26.0
4.2).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 163
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ gcc -version
gcc: error: unrecognized command-line option '-version'
gcc: fatal error: no input files
compilation terminated.
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  T
here is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICUL
AR PURPOSE.

ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ |
```

Figure 4: Installation of GCC and Version check

Summary: Installed the build-essential package using apt and verified whether its version is up to date or not. It has the latest version of GCC.

B. Four Stages of Compilation



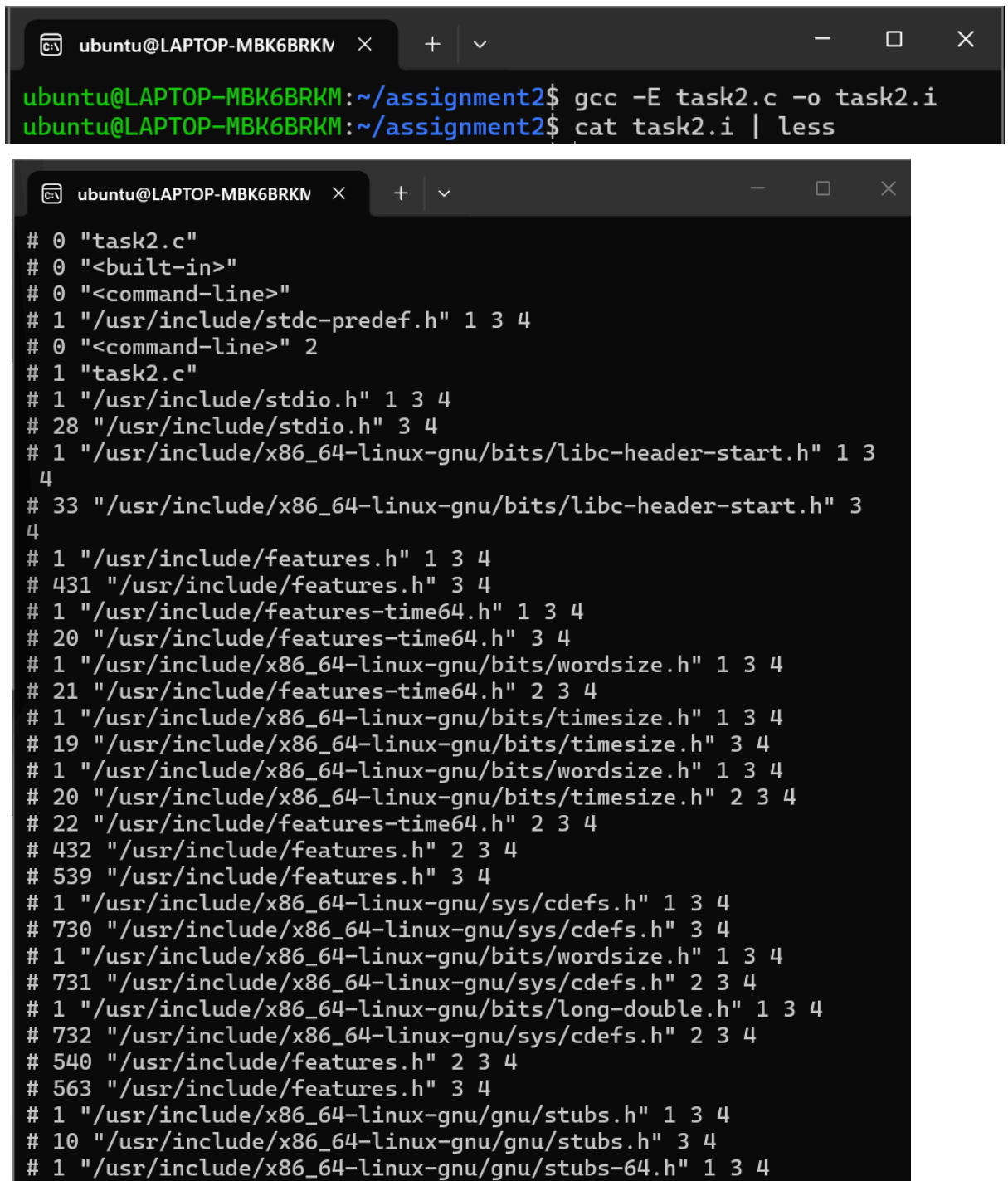
```
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ gcc -E task2.c -o task2.i
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ gcc -S task2.i -o task2.s
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ gcc -c task2.s -o task2.o
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ gcc task2.o -o task2
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ ./task2
Enter your name: santosh
Enter your age: 18
Enter your height (m): 5.7

--- User Info ---
Name: santosh
Age: 18 years
Height: 5.70 m
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ |
```

Figure 5: Using 4 distinct stages of C compilation

Summary: Wrote 4 stages of compilation where preprocessing, compiling, assembling, and linking are done by completing and running the code using (.). The interaction was also done here, as you can see.

C. COMPILATION STAGE EXAMPLES



The image consists of two terminal window screenshots. The top screenshot shows the execution of the GCC compiler with the `-E` flag to perform preprocessing. The command `gcc -E task2.c -o task2.i` is entered, followed by `cat task2.i | less` to view the output. The bottom screenshot shows the output of the preprocessing stage, which lists the source files and headers included in the compilation, along with line numbers. The output is as follows:

```
# 0 "task2.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "task2.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3
4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3
4
# 1 "/usr/include/features.h" 1 3 4
# 431 "/usr/include/features.h" 3 4
# 1 "/usr/include/features-time64.h" 1 3 4
# 20 "/usr/include/features-time64.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 21 "/usr/include/features-time64.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 1 3 4
# 19 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 20 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 2 3 4
# 22 "/usr/include/features-time64.h" 2 3 4
# 432 "/usr/include/features.h" 2 3 4
# 539 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 1 3 4
# 730 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 731 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/long-double.h" 1 3 4
# 732 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 540 "/usr/include/features.h" 2 3 4
# 563 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 1 3 4
# 10 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs-64.h" 1 3 4
```

Figure 5: Preprocessing Stage

```
ubuntu@LAPTOP-MBK6BRKM x + v - □ ×
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ gcc -S task2.i -o task2.s
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ cat task2.s
.file "task2.c"
.text
.section .rodata
.LC0:
.string "Enter your name: "
.LC1:
.string "%s"
.LC2:
.string "Enter your age: "
.LC3:
.string "%d"
.LC4:
.string "Enter your height (m): "
.LC5:
.string "%f"
.LC6:
.string "\n--- User Info ---"
.LC7:
.string "Name: %s\n"
.LC8:
.string "Age: %d years\n"
.LC9:
.string "Height: %.2f m\n"
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
endbr64
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
subq $80, %rsp
movq %fs:40, %rax
movq %rax, -8(%rbp)
xorl %eax, %eax
leaq .LC0(%rip), %rax
movq %rax, %rdi
movl $0, %eax
call printf@PLT
leaq -64(%rbp), %rax
leaq .LC1(%rip), %rdx
movq %rax, %rsi
movq %rdx, %rdi
```

Figure 6: Compilation Stage

```
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ gcc -c task2.s -o task2.o
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ file task2.o
task2.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not st
ripped
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ |
```

Figure 7: Assembly Stage

```
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ gcc task2.o -o task2
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ ./task2
Enter your name: santosh
Enter your age: 18
Enter your height (m): 5.7

--- User Info ---
Name: santosh
Age: 18 years
Height: 5.70 m
ubuntu@LAPTOP-MBK6BRKM:~/assignment2$ |
```

Figure 8: Linking Stage

IV. Return Values and Exit Status

A. Understanding Return Values

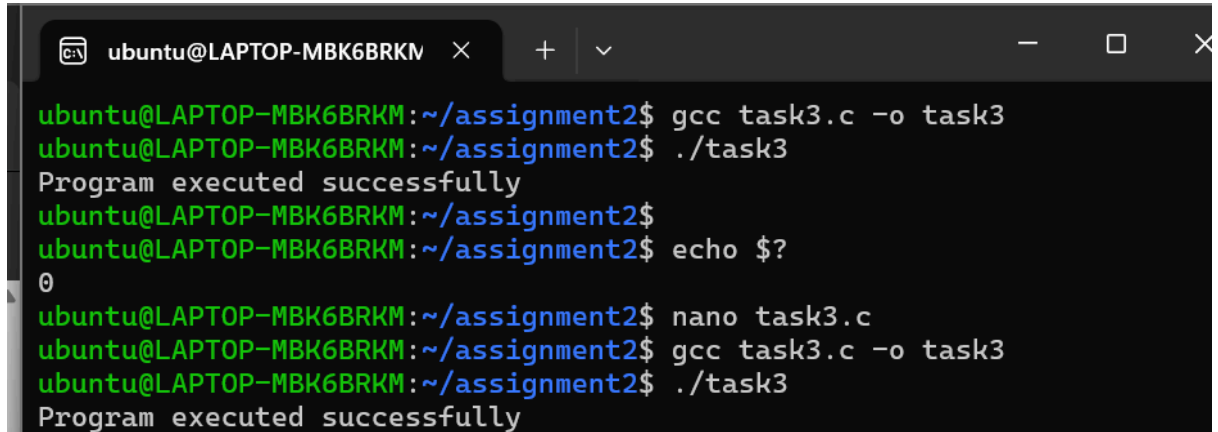
A terminal window titled 'ubuntu@LAPTOP-MBK6BRKM' showing a series of commands and their outputs. The user compiles 'task3.c' into 'task3' using 'gcc', runs './task3', which outputs 'Program executed successfully'. Then, the user runs 'echo \$?' which outputs '0', indicating successful execution. Finally, the user edits 'task3.c' with 'nano', recompiles, and runs the program again, which also outputs 'Program executed successfully'.

Figure 9: Program with return value

B. Checking Exit Status

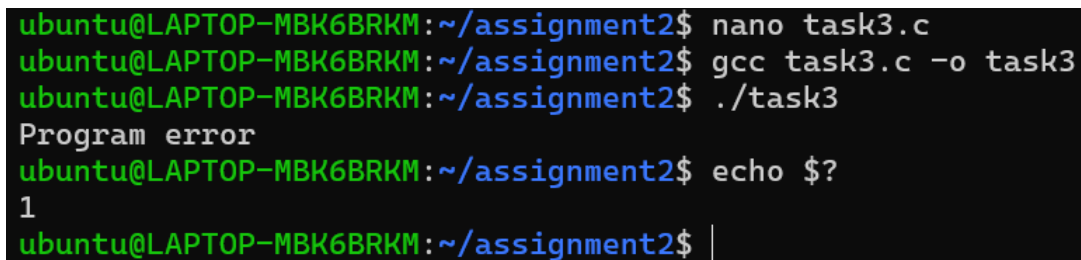
A terminal window titled 'ubuntu@LAPTOP-MBK6BRKM' showing a series of commands. The user compiles 'task3.c' into 'task3' using 'gcc', runs './task3', which outputs 'Program error'. Then, the user runs 'echo \$?' which outputs '1', indicating an error. The terminal ends with a cursor on a new line.

Figure 10: Checking Program Exit Status

Summary: Wrote code in task3.c, used gcc, and (./) ran the code and executed as planned. Additionally, if the result was “Program executed successfully,” then using the echo \$? Then the result value should be “1”. Such that when the result is “Program error,” then it should print out the value as “0.”

V. KNOWLEDGE CHECK QUESTIONS

A. Question 1

Ans) The `#include` directive is used to include a header file and represents a preprocessor directive in a C program.

Example,

Using `#include <stdio.h>`, it allows to use the function such as `printf()` and `scanf()`.

B. Question 2

Ans) Preprocessing (`gcc -E`) expands macros and `#include` directives, producing a `.i` file. Compilation (`gcc -S`) translates the preprocessed C code into assembly language, producing a `.s` file. Assembly (`gcc -c`) converts the assembly code into machine code stored in an object file with a `.o` extension. Linking (`gcc file.o -o program`) combines the object file with any required libraries and startup code to produce the final executable, which has no extension on Linux (e.g., `hello`).

C. Question 3

Ans) A return value of 0 tells the operating system that the program finished successfully with no errors. A non-zero return value signals that the program terminated abnormally or encountered a specific error condition, and the particular number can be used by scripts or the calling process to identify what went wrong. This was demonstrated directly: `hello` returned 0 after normal execution, and `echo $?` reported 50 once `main()` was changed to return 50.

D. Question 4

Ans) (1) `gcc -E file.c -o file` stops after preprocessing. (2) `gcc -S file.c -o file.s` stops after generating assembly code. (3) `gcc -c file.c -o file.o` stops after assembling, producing an object file without linking it into an executable.

E. Question 5

Ans) C is called a compiled language because its source code is translated in advance, by GCC in this lab, through the preprocessing, compilation, assembly, and linking stages into a standalone machine-code executable that the CPU can run directly. An interpreted language, by contrast, has no separate build step: an interpreter reads and executes the source code (or a lightweight bytecode form of it) one statement at a time at runtime, which is generally slower but skips the compile-then-run workflow seen throughout this lab.